

Feedforward Neural Network Classification on CIFAR-10: Performance Analysis and Comparison with kNN and NCC

Elisavet Stougiannou

Graduate Student of Electrical and Computer Engineering Department,
Aristotle University of Thessaloniki

Email: estougia@ece.auth.gr

Abstract— This project focuses on training and evaluating a feedforward neural network for multi-class image classification using the CIFAR-10 dataset. The model was implemented in Python with PyTorch and trained using back-propagation. Several experiments were conducted by varying the number of hidden neurons and the learning rate in order to explore how these parameters influence accuracy, generalization, and training time. The best configuration achieved approximately 96% training accuracy and around 76% test accuracy. In addition, the performance of the neural network was compared with traditional baseline classifiers, namely k-Nearest Neighbors and Nearest Class Centroid, implemented earlier in the semester. The neural network clearly outperformed both methods, confirming the advantage of learned representations over raw distance-based classification. The report summarizes the methodology, presents the experimental results, and discusses the strengths and limitations of the implemented model.

I. INTRODUCTION

Artificial Neural Networks (ANNs) constitute one of the fundamental approaches in modern machine learning, inspired by the way biological neurons process information. A typical neural network is composed of layers of interconnected units that transform input data through weighted connections and nonlinear activation functions. By adjusting these weights during training, the network gradually learns patterns that allow it to perform tasks such as classification or regression.

Convolutional Neural Networks (CNNs) extend this idea by introducing convolutional layers, which are especially effective when dealing with images. Instead of receiving raw pixel values as a flat vector, CNNs exploit spatial structure by applying filters that detect local features such as edges, textures, and shapes. This hierarchical feature extraction allows deeper layers to learn increasingly complex representations and has made CNNs the standard choice for computer vision problems.

The purpose of this project is to explore image classification on the CIFAR-10 dataset using a custom neural network implemented in PyTorch. The work focuses on understanding how architectural choices and training parameters affect

performance. To this end, several experiments were carried out, varying the number of neurons in the hidden layer and the learning rate. The resulting model is also compared against classical machine-learning methods, k-Nearest Neighbors and Nearest Class Centroid, in order to highlight the differences between traditional classifiers and learned deep representations. Through this process, the project aims to provide both practical experience with neural-network training and insight into the factors that influence model accuracy and generalization.

II. DATASET

The experiments in this project were conducted using the CIFAR-10 dataset, one of the most widely used benchmarks in image-classification research. CIFAR-10 consists of 60,000 color images of size 32×32 pixels, split into 50,000 training samples and 10,000 test samples. Each image belongs to one of 10 object categories: **airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck**.

The dataset is intentionally challenging due to its small image resolution and the high variability within each class. Objects may appear in different poses, lighting conditions, and backgrounds, which forces models to learn robust and generalizable features. All images were loaded using the **torchvision.datasets**. CIFAR10 utility, and a simple preprocessing pipeline was applied. Specifically, each image was converted into a tensor and normalized channel-wise using mean and standard deviation values centered around 0.5. No data augmentation techniques (such as flips, crops, or rotations) were used in this project, in order to focus strictly on the effect of the model architecture and training parameters.

Overall, CIFAR-10 provides an appropriate balance between simplicity and complexity, making it suitable for exploring neural-network behavior while keeping training times manageable.

III. METHODOLOGY

The methodology followed in this project was structured around two main goals: first, to design and train a convolutional neural network (CNN) for CIFAR-10

Neural Networks and Deep Learning – 1st Project

classification, and second, to systematically evaluate how architectural choices and training parameters affect performance.

A. Data Processing

All images were normalized to have pixel values centered around zero, following the standard transformation:

- Conversion to tensor
- Channel-wise normalization with mean = (0.5, 0.5, 0.5) and std = (0.5, 0.5, 0.5)

B. Model Architecture

The implemented CNN consisted of two convolutional blocks followed by a fully connected classification head. Batch normalization and ReLU activations were included in each block to stabilize training. Max pooling was applied after each convolutional stage to progressively reduce spatial dimensions.

The final feature map was flattened and passed through a fully connected layer whose size varied across experiments (128, 256, 384, 512, 1024 neurons). A final linear layer mapped the feature vector to 10 output classes.

This structure was chosen because it is simple enough for controlled experimentation, yet expressive enough to achieve competitive accuracy on CIFAR-10.

C. Training Procedure

The model was trained using:

- Cross-Entropy Loss as the objective function
- Adam as the optimizer
- Learning rates: 0.0005 and 0.001
- Batch size: 64
- Training duration: 10 epochs for each configuration

For every combination of learning rate and number of neurons, the following metrics were recorded:

- Training accuracy per epoch
- Test accuracy per epoch
- Training time
- Final confusion matrix
- Example correct and incorrect predictions

This setup allowed for a direct comparison between architectures and training settings.

D. Baseline Models

To complement the CNN experiments, several classical models were also evaluated:

- **k-Nearest Neighbors (kNN)** with $k=1$ and $k=3$
- **Nearest Class Centroid (NCC)**

Both **with** and **without PCA** dimensionality reduction.

Although these models perform significantly worse than CNNs on raw images, they offer a useful reference point and highlight the advantages of learning-based feature extraction.

IV. RESULTS

This section presents the experimental results obtained from training the CNN model across multiple architectural configurations and learning rates. The evaluation includes training and test accuracy, training time, confusion matrices, and qualitative examples of correct and incorrect predictions. The performance of the classical methods from the intermediate project (kNN and NCC) is also included for comparison.

A. Quantitative Results

- CNN Performance Across Hidden Layer Sizes (Learning Rate = 0.0005)

Hidden Units	Train Accuracy (%)	Test Accuracy (%)	Training Time (sec)
128	~87	73.59	225.85
256	~90	73.84	249.68
384	~94	73.96	264.22
512	~94	74.77	265.30
1024	~95	75.95	324.96

- CNN Performance Across Hidden Layer Sizes (Learning Rate = 0.001)

Hidden Units	Train Accuracy (%)	Test Accuracy (%)	Training Time (sec)
128	~87	74.42	214.64
256	~91	74.23	250.59
384	~93	73.44	262.32
512	~93	74.63	326.66
1024	~94	74.91	326.04

Although increasing the learning rate sped up training slightly at smaller layers, it did not lead to significant improvement in final accuracy. The highest accuracy was

Neural Networks and Deep Learning – 1st Project

again achieved by the **1024 neurons model**, but slightly lower compared to the 0.0005 configuration.

- Classical Models

Model	Accuracy (%)	Time (s)	Notes
kNN (k=1)	35.39	8.63	No PCA
kNN (k=3)	33.03	7.33	No PCA
NCC	27.74	1.24	No PCA
kNN (k=1)	38.73	0.40	With PCA
kNN (k=3)	36.72	0.39	With PCA
NCC	27.68	0.05	With PCA

These results demonstrate that even a simple CNN vastly outperforms traditional nearest-neighbor methods on raw images, both with and without PCA.

B. Training Curves

Two main plots were generated for each experiment:

- Training Loss per Epoch:

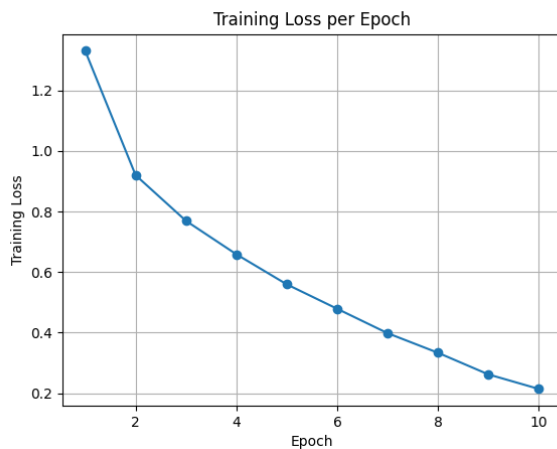


Fig.1. Training Loss per Epoch-1024 neurons

A line plot showing the decrease of training loss across the 10 epochs.

All models showed stable convergence with no oscillations, indicating that the chosen learning rates were appropriate.

- Training vs Test Accuracy per Epoch:

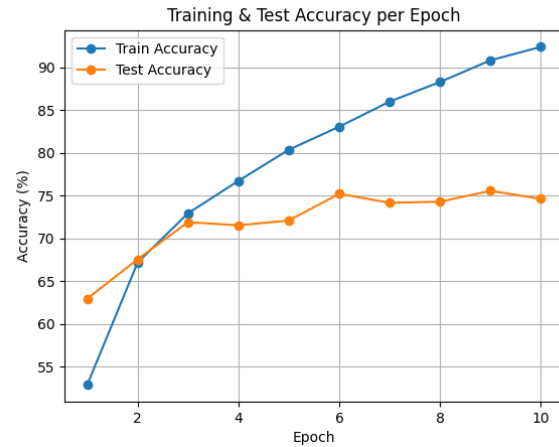


Fig.2. Training & Test Accuracy per Epoch-1024 neurons

These plots visualize the progression of training and test accuracy.

A consistent gap can be observed between training and test curves.

C. Confusion Matrices

For each configuration, a confusion matrix was generated using the final model predictions on the test set.

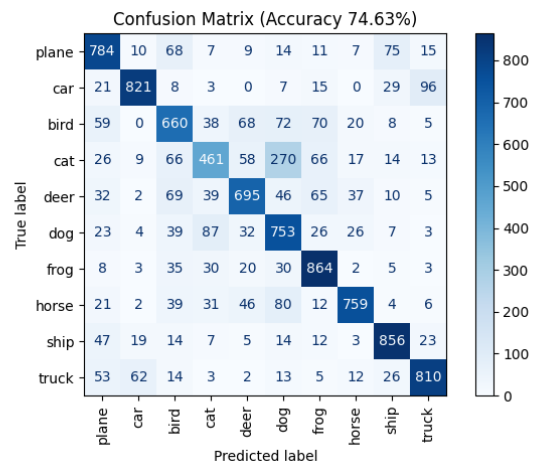


Fig.3. Confusion Matrix-1024 neurons & lr=0.0005

In the best-performing configuration (1024 neurons, lr = 0.0005), the confusion matrix shows:

- Very strong performance on “**automobile**”, “**ship**”, and “**truck**”
- More confusion between “**cat**”, “**dog**”, “**deer**”, and other animal classes
- Misclassifications mostly occur between semantically similar classes, which aligns with well-known CIFAR-10 difficulty patterns.

Neural Networks and Deep Learning – 1st Project

D. Qualitative Results (Correct & Incorrect Predictions)

To better visualize the model's behavior, sample images from the test set were saved into two groups:

- Correct Predictions

These usually include:

- High-contrast images
- Clear object shapes
- Classes with distinctive textures (e.g., ship, truck, airplane)



Fig.4. Correct Predictions-1024 neurons

- Incorrect Predictions

Most incorrect predictions come from:

- Blurry or low-contrast images
- Occluded objects
- Classes with overlapping visual patterns (e.g., cat ↔ dog, deer ↔ horse)

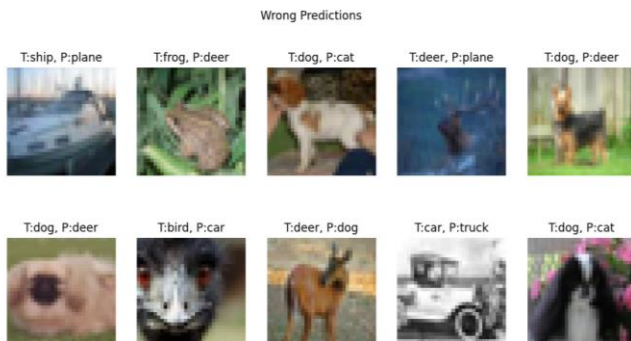


Fig.5. Wrong Predictions-1024 neurons

These qualitative examples highlight limitations of the model and complement the confusion matrix findings.

E. Summary of Findings

- The final best model achieved 75.95% test accuracy.
- Performance improved consistently as the hidden layer size increased.
- Lower learning rate (0.0005) produced better results overall.
- CNNs significantly outperform classical methods such as kNN and NCC on image data.
- Class confusions follow expected patterns for CIFAR-10.

V. DISCUSSION

This section summarizes and interprets the results of the classification methods applied to CIFAR-10. The focus is on understanding the strengths and weaknesses of each approach, explaining why their performance differs, and interpreting the observed outcomes.

Overall Performance Overview

The three methods show clear differences in architecture, complexity, and generalization ability:

1. **Neural Network (NN):**
Achieved the highest accuracy among the three methods.
Its ability to learn complex feature representations allows it to distinguish visually diverse and noisy images more effectively.
2. **K-Nearest Neighbors (KNN):**
Performance strongly depends on the choice of k and the distance metric.
Performs better than NCC but clearly worse than the NN.
3. **Nearest Centroid Classifier (NCC):**
Reached the lowest accuracy, which is expected since each class is represented by a **single mean vector**, inadequate for modeling the complexity of CIFAR-10 images.

VI. DISCUSSION

Through the experiments conducted in this project, several important observations emerged regarding the effectiveness of different classification approaches on the CIFAR-10 dataset.

First, the results clearly show that **neural networks outperform classical methods** like KNN and NCC. The neural network achieved the highest test accuracy across all configurations, demonstrating its ability to learn meaningful and hierarchical representations of images. Its performance improved consistently with larger hidden layers, up to the best-performing configuration of **1024 neurons**, confirming

Neural Networks and Deep Learning – 1st Project

that additional model capacity helps capture the complexity of natural images.

In contrast, **KNN** provided moderate accuracy and proved sensitive to the choice of k . While its performance slightly improved when PCA was applied, it still struggled with the high dimensionality of raw images and resulted in slow predictions, making it unsuitable for large-scale image tasks.

Finally, **NCC** served as a simple baseline but delivered the lowest accuracy. Representing each class with a single centroid is too restrictive for datasets with large intra-class variability like CIFAR-10. However, its extremely fast computation highlights why it is often used as a theoretical reference rather than a practical classifier.

Overall, the project demonstrates that **learning-based models are far more effective for image classification** than distance-based methods operating directly on pixels. The experiments also highlight the importance of model capacity and the influence of training parameters such as the learning rate.

In summary, what we learned is that **neural networks constitute the most appropriate and scalable solution** for the CIFAR-10 problem, while KNN and NCC provide useful comparisons that help illustrate the strengths of modern deep learning approaches.

VII. REFERENCES

- [1] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [3] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.